Groups, coding theory and Pólya counting

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.1 Group axioms and uniqueness

A group is a nonempty set G with a binary operation $G \times G \to G$, associative multiplication, a two-sided identity e, and a two-sided inverse a^{-1} for every a. Closure is included in the operation's codomain.

If e, e' are identities, $e = ee' = e'$. If b, c are inverses of a, then

$$b = eb = (ca)b = c(ab) = ce = c.$$

Left cancellation follows by multiplying $ab = ac$ on the left by a^{-1}. Right cancellation follows similarly.

Associativity changes parentheses, not factor order. In general $(ab)^{-1} = b^{-1}a^{-1}$, because multiplication by this expression on either side gives e.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.1.1 Permutations act on the right first

Use $(\sigma\tau)(i) = \sigma(\tau(i))$. A cycle $(a_1 \cdots a_r)$ sends each point to the next and the last to the first, fixing unlisted points. Disjoint cycles commute. For $g = (13), h = (12)$:

i	$h(i)$	$g(h(i))$	$g(i)$	$h(g(i))$
1	2	2	3	3
2	1	3	2	1
3	3	1	1	2

Thus $gh = (123)$, whereas $hg = (132)$. An r-cycle raised to k fixes all its points exactly when $r \mid k$. For disjoint cycle lengths $r_1, \ldots, r_s$, the permutation order is their lcm. For example, $(123)(45)$ has order six, not five. Operation order also matters for left and right cosets.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.1.2 Boundaries for matrix and residue multiplication

The invertible real $n \times n$ matrices form $GL_n(\mathbb{R})$. For $n = 2$, take

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Both determinants are 1, but $AB \neq BA$. For $n > 2$, block-diagonal extension with I_{n-2} preserves this failure. Thus GL_n is nonabelian for $n \geq 2$, whereas $GL_1(\mathbb{R}) \cong \mathbb{R}^\times$ is abelian.

For every $n \geq 2$, all of $\mathbb{Z}_n$ under multiplication is not a group: $[0]$ has no inverse. For prime p, the nonzero classes $\mathbb{Z}_p^\times$ do form a multiplicative group by Bezout.

A multiplicative monoid containing zero is different from its group of units, even at a prime modulus.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.1.3 Subgroups and orders

A nonempty subset $H \subseteq G$ is a subgroup exactly when $a, b \in H \Rightarrow ab^{-1} \in H$. Setting $a = b$ gives the identity, then taking $a = e$ gives inverses, and applying the test to a, b^{-1} gives products.

Group order means $|G|$. Element order is the least positive m with $g^m = e$, or infinity if none exists. In $(\mathbb{Z}_8, +)$, multiples $2, 4, 6, 0$ show that 2 has order 4 and generates $\{0, 2, 4, 6\}$.

In a finite group, powers repeat, giving a positive power equal to the identity. Infinite order occurs, for example, for $1 \in (\mathbb{Z}, +)$. Lagrange will imply that finite element orders divide group order. The two meanings of order must remain distinct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2 Homomorphisms, kernels, and images

A homomorphism $\varphi : G \to H$ satisfies $\varphi(ab) = \varphi(a)\varphi(b)$. It sends identity to identity, because the idempotent $\varphi(e)$ cancels in a group, and sends inverses to inverses.

The kernel $\{g : \varphi(g) = e_H\}$ and image are subgroups, by the subgroup test. Injectivity is equivalent to a trivial kernel. Indeed,

$$\varphi(a) = \varphi(b) \iff \varphi(a^{-1}b) = e_H.$$

A trivial kernel forces $a = b$; injectivity conversely makes the identity's preimage unique.

Preserving an operation does not by itself imply injectivity, surjectivity, or equal cardinalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2.1 Cyclic groups and their images

The subgroup generated by g is $\langle g \rangle = \{g^k : k \in \mathbb{Z}\}$. A cyclic group equals such a subgroup and is abelian, since $g^m g^n = g^{m+n} = g^n g^m$. A finite cyclic group of order n is isomorphic to $\mathbb{Z}_n$ through $[k] \mapsto g^k$.

If $G = \langle g \rangle$, then $\varphi(G) = \langle \varphi(g) \rangle$. The map $\mathbb{Z}_{18} \to \mathbb{Z}_6$, $[k]_{18} \mapsto [k]_6$, is well defined, has kernel $\{0, 6, 12\}$, and is onto.

The Klein four-group $\mathbb{Z}_2 \times \mathbb{Z}_2$ is abelian but not cyclic, since every nonidentity element has order 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2.2 Classification and subgroups of cyclic groups

The map $\psi : \mathbb{Z} \to \langle g \rangle, k \mapsto g^k$, is a surjective homomorphism. If g has infinite order, different integers give different powers, so it is an isomorphism. If $o(g) = n$, division with remainder gives $g^k = e \iff n \mid k$. Thus $[k]_n \mapsto g^k$ is a well-defined bijective homomorphism from $\mathbb{Z}_n$.

Every subgroup H of a cyclic group is cyclic. The trivial case is immediate. Otherwise the exponents of elements of H include a positive integer. Let d be the least positive one. For any $g^k \in H$, write $k = qd + r, 0 \leq r < d$. Then $g^r = g^k (g^d)^{-q} \in H$, so minimality forces $r = 0$. Therefore $H = \langle g^d \rangle$. In the finite case the trivial subgroup also has this form with $d = n$.

If $o(g) = n < \infty$, then

$$o(g^k) = \frac{n}{\gcd(n, k)}.$$

Indeed, putting $d = \gcd(n, k)$ gives $n \mid kt \iff n/d \mid t$, because n/d and k/d are coprime. The least positive t is n/d. Thus g^k generates the whole group exactly when $\gcd(n, k) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2.3 All homomorphisms between finite cyclic groups

An additive homomorphism $\mathbb{Z}_m \to \mathbb{Z}_n$ is determined by $t = \varphi([1]_m)$, and must satisfy

$$\varphi([a]_m) = [at]_n, \qquad mt \equiv 0 \pmod{n}.$$

Necessity follows from $m[1]_m = 0$. Conversely, when $mt \equiv 0$, changing a representative by qm changes its image by $qmt \equiv 0$, so the map is well defined and additive.

For $d = \gcd(m, n)$, all solutions are $t = j(n/d)$, $j = 0, \ldots, d-1$. There are exactly d homomorphisms. The image is $\langle [t]_n \rangle$, with size $n/\gcd(n, t)$. Equal-sized fibers give kernel size $m \gcd(n, t)/n$. Surjectivity requires $\gcd(n, t) = 1$ and $n \mid m$. Cyclicity of both groups alone is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3 Cosets and equality

For $H \leq G$, define left and right cosets

$$aH = \{ah : h \in H\}, \qquad Ha = \{ha : h \in H\}.$$

Unless specified otherwise, use left cosets. The map $h \mapsto ah$ is a bijection $H \to aH$, with inverse $x \mapsto a^{-1}x$. Thus every coset has the same cardinality as H.

$$aH = bH \iff b^{-1}a \in H, \qquad Ha = Hb \iff ab^{-1} \in H.$$

For the first identity, equal cosets put a in bH; conversely $a = bh$ gives $aH = bH$.

Since $e \in H$, every a lies in its own coset. Left and right cosets coincide for abelian groups, but not in general.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.1 Why cosets partition the group

If $aH \cap bH \neq \varnothing$, choose $x = ah_1 = bh_2$. Then $a = bh_2h_1^{-1}$, and for every $h \in H$, $ah = b(h_2h_1^{-1}h) \in bH$. Hence $aH \subseteq bH$, and symmetry gives equality.

Distinct cosets are therefore disjoint. Every $g \in G$ belongs to gH, so

$$G = \bigsqcup_{aH \in G/H} aH.$$

For finite G, if there are k cosets, then $|G| = k|H|$.

No normality assumption enters this partition argument. Normality is required for quotient multiplication, not for the coset set itself.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.2 Lagrange and its failed converse

For finite G and $H \leq G$,

$$[G : H] = \frac{|G|}{|H|}, \qquad |H| \mid |G|.$$

Apply this to $\langle a \rangle$ to obtain $o(a) \mid |G|$, hence $a^{|G|} = e$.

The converse fails: A_4 has order twelve but no subgroup of order six. A full proof follows after quotient groups are established. Divisibility alone is a necessary subgroup-size condition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.3 Normality

A subgroup H is normal, written $H \trianglelefteq G$, if for every $g \in G$,

$$gH = Hg \iff gHg^{-1} = H \iff g^{-1}Hg = H.$$

Equivalently, $g^{-1}hg \in H$ for every $g \in G, h \in H$; applying inclusion also to g^{-1} yields equality.

Kernels are normal, since

$$\varphi(g^{-1}kg) = \varphi(g)^{-1}\varphi(k)\varphi(g) = e$$

for $k \in \ker \varphi$. Every subgroup of an abelian group is normal.

Normality is preservation under conjugation. It is not a visual symmetry condition on a subgroup diagram.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.4 Representative-independent quotient multiplication

Try to define $(aH)(bH) = (ab)H$. If H is normal and $a' = ah_1, b' = bh_2$, then $h_1 b = bh_3$ for some $h_3 \in H$, so

$$a'b' = ah_1 bh_2 = abh_3 h_2 \in abH.$$

Thus $(a'b')H = (ab)H$, independent of representatives.

Conversely, if multiplication is well defined, replacing representative e of H by $h \in H$ gives $(hg)H = gH$. Hence $hg = gh'$ and $g^{-1}hg = h' \in H$ for every h, g, establishing normality. Associativity then descends from G, identity is H, and inverse of aH is $a^{-1}H$.

Quotient operations act on representatives first. Independence is a necessary mathematical obligation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.3.5 A nonnormal subgroup of S_3

Use right-to-left permutation composition. Let $H = \{e, (12)\}$ and $g = (13)$. Then

$$gH = \{(13), (123)\}, \qquad Hg = \{(13), (132)\}.$$

The sets differ, so H is not normal. The same left coset has representatives (13) and (123). Squaring the first gives $e \in H$, but squaring the second gives $(132) \notin H$. The proposed product of that coset with itself changes with the representatives.

A subgroup always gives cosets, but only a normal subgroup gives this quotient group operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.6 The first isomorphism theorem

Let $\varphi : G \to K$ be a homomorphism and $N = \ker \varphi \trianglelefteq G$. The quotient operation is well defined. Associativity descends from representative multiplication, the identity is N, and the inverse of gN is $g^{-1}N$, so G/N is a group.

Define $\bar{\varphi} : G/N \to \operatorname{Im} \varphi$ by $\bar{\varphi}(gN) = \varphi(g)$. Since

$$gN = hN \iff h^{-1}g \in N \iff \varphi(g) = \varphi(h),$$

the map is both well defined and injective. It is onto by the definition of image. Also

$$\bar{\varphi}((gN)(hN)) = \varphi(gh) = \varphi(g)\varphi(h).$$

Thus $G/\ker \varphi \cong \operatorname{Im} \varphi$. Replacing the image by K requires a separate surjectivity proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.7 Why the converse to Lagrange fails

The even permutation group A_4 has order $4!/2 = 12$. Suppose H were a subgroup of order 6. Its index would be 2. For every $g \notin H$, both gH and Hg equal $G \setminus H$, so every index-2 subgroup is normal. The natural projection to A_4/H is therefore a homomorphism to a group of order 2.

For any 3-cycle a, its image has order dividing 3 because $a^3 = e$, and also dividing 2 by Lagrange in the quotient. Its order is therefore 1, so every 3-cycle lies in H.

There are $\binom{4}{3} \cdot 2 = 8$ distinct 3-cycles on four points. Each is even because $(abc) = (ac)(ab)$. Together with the identity they force at least nine elements in H, contradicting $|H| = 6$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.4 RSA key generation

Choose distinct primes p, q, set $n = pq$, and compute $\varphi(n) = (p-1)(q-1)$. Choose a positive public exponent e coprime to $\varphi(n)$, and use extended Euclid to choose a positive d with $ed \equiv 1 \pmod{\varphi(n)}$.

The public key is (n, e), and the private exponent is d. For a representative $0 \le M < n$,

$$C = M^e \bmod n, \qquad M' = C^d \bmod n.$$

Positive exponents keep powers defined on nonunit messages. This teaching model proves the number-theoretic inverse property. The formulas are deterministic, giving the same ciphertext for the same message, and include neither randomized encoding nor a complete communication protocol.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.4.1 Correctness for coprime messages

Write $ed = 1 + k(p-1)(q-1)$. If $\gcd(M, n) = 1$, Euler's theorem gives $M^{\varphi(n)} \equiv 1 \pmod{n}$, hence

$$(M^e)^d = M^{ed} = M\left(M^{(p-1)(q-1)}\right)^k \equiv M \pmod{n}.$$

This proves decryption for coprime messages, but alone says nothing about representatives divisible by p or q.

The message space contains every residue class modulo n. Correctness must include noncoprime classes as well.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.4.2 CRT proves correctness for every message

Work modulo p. If $M \equiv 0 \pmod{p}$, then $M^{ed} \equiv M$. Otherwise Fermat gives $M^{p-1} \equiv 1 \pmod{p}$, and since $p - 1 \mid ed - 1$,

$$M^{ed} = M(M^{p-1})^{(ed-1)/(p-1)} \equiv M \pmod{p}.$$

The same cases apply modulo q. Thus both primes divide $M^{ed} - M$. Their coprimality and CRT imply $M^{ed} \equiv M \pmod{pq}$.

This argument includes zero and every message sharing a factor with n, without misapplying Euler's coprime hypothesis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.4.3 Complete example with $M = 7$

Choose $p = 5, q = 11, e = 17$. Then $n = 55$ and $\varphi(n) = 40$. Euclid gives

$$40 = 2(17) + 6, \quad 17 = 2(6) + 5, \quad 6 = 5 + 1,$$

$$1 = 6 - 5 = 3 \cdot 6 - 17 = 3(40 - 2 \cdot 17) - 17 = 3 \cdot 40 - 7 \cdot 17.$$

Thus $d = 33$. Repeated squares modulo 55 are

$$7^2 \equiv 49, \quad 7^4 \equiv 36, \quad 7^8 \equiv 31, \quad 7^{16} \equiv 26,$$

so $C \equiv 26 \cdot 7 \equiv 17$. For decryption,

$$17^2 \equiv 14, \quad 17^4 \equiv 31, \quad 17^8 \equiv 26, \quad 17^{16} \equiv 16, \quad 17^{32} \equiv 36,$$

$$17^{33} \equiv 36 \cdot 17 = 612 \equiv 7 \pmod{55}.$$

Reduce after every square and multiplication instead of expanding enormous integer powers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.4.4 Complete example with noncoprime $M = 15$

Keep $n = 55, e = 17, d = 33$. Since $\gcd(15, 55) = 5$, use CRT. Encryption is zero modulo 5. Modulo 11, $15 \equiv 4$ and

$$4^{17} \equiv 4^7, \qquad 4^2 \equiv 5, \quad 4^4 \equiv 3, \quad 4^7 \equiv 3 \cdot 5 \cdot 4 \equiv 5.$$

Thus $C \equiv 0 \pmod 5$, $C \equiv 5 \pmod{11}$, giving $C = 5$.

For decryption, $5^{33} \equiv 0 \pmod 5$, while $5^{33} \equiv 5^3 \equiv 4 \pmod{11}$. Write $M' = 5k$. Since $5^{-1} \equiv 9 \pmod{11}$, $k \equiv 9 \cdot 4 \equiv 3$, giving $M' \equiv 15 \pmod{55}$.

This explicitly verifies a noncoprime message. A successful small arithmetic example establishes no cryptographic security.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.5 Detection and correction guarantees

A length-n block code is $C \subseteq \Sigma^n$, binary when $\Sigma = \{0, 1\}$. Codes here are nonempty. For a singleton set $d(C) = +\infty$, using the empty-pair convention. For at least two codewords, minimum distance is

$$d(C) = \min_{c \neq c'} d(c, c').$$

If $d(C) \geq k + 1$, membership testing detects every nonzero error of weight at most k: its received word lies less than minimum distance from the sent codeword and cannot be a different codeword.

If $d(C) \geq 2t + 1$, radius-t balls around codewords are disjoint. Otherwise one received word in two balls would give $d(c_1, c_2) \leq 2t$ by the triangle inequality. Nearest-codeword decoding therefore uniquely corrects every error of weight at most t.

Detection and correction describe different decoder behavior. Their separate maximum bounds cannot automatically be combined in one protocol.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.5.1 The distance thresholds are necessary

If two codewords have distance $d \leq k$, an error of weight d changes one directly into the other. A membership detector accepts the received codeword, so it cannot detect every nonzero error of weight at most k.

If $d(c_1, c_2) = d \leq 2t$, construct r by taking the coordinates of c_2 at $\lfloor d/2 \rfloor$ of their differing positions and those of c_1 elsewhere. Then

$$d(r, c_1) = \lfloor d/2 \rfloor \leq t, \qquad d(r, c_2) = \lceil d/2 \rceil \leq t.$$

The same received word can arise from two different transmitted codewords after at most t errors. No decoder can always recover the transmitted word uniquely. Thus $d \geq 2t + 1$ is necessary for the worst-case correction guarantee, not merely sufficient for one algorithm.

Average error rates on a specified probabilistic channel are a separate question.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.5.2 Why the two guarantees cannot be added

The repetition code $C = \{000, 111\}$ has distance 3. Membership detection detects every one- or two-bit error. Majority decoding corrects every one-bit error. But sending 111 and receiving 100 after two flips gives distance 1 from 000 and distance 2 from 111, so correction misdecodes it as 000.

For a specified correction radius t and additional detection band $s \geq 0$, a bounded-distance decoder can correct errors through t and flag weights $t + 1$ through $t + s$ when $d \geq 2t + s + 1$. If such a heavier received word were within t of another codeword, the codeword distance would be at most $2t + s$, a contradiction.

State the channel model, correction radius, detection band, and behavior outside them. A code's distance is not an unconditional transmission guarantee.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.6 Hamming distance is a metric

For binary words,

$$wt(x) = |\{i : x_i = 1\}|, \qquad d(x, y) = |\{i : x_i \neq y_i\}| = wt(x + y).$$

Over $\mathbb{F}_2$, a coordinate sum is 1 exactly when the bits differ. Nonnegativity, symmetry, and $d(x, y) = 0 \iff x = y$ follow coordinatewise.

For the triangle inequality,

$$1[x_i \neq z_i] \leq 1[x_i \neq y_i] + 1[y_i \neq z_i].$$

Summing gives $d(x, z) \leq d(x, y) + d(y, z)$.

This metric counts differing positions. Erasures, weighted substitutions, or other channel models require different definitions or guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.6.1 Balls and sphere packing

The binary radius-t ball is $B(x,t)=\{y : d(x,y)\le t\}$, of size

$$V(n,t)=\sum_{j=0}^{\min(t,n)}\binom{n}{j}\qquad (t \text{ a nonnegative integer}).$$

Choose the j positions flipped from x. For $t\ge n$, the ball is the whole space. Sums written up to t use $\binom{n}{j}=0$ for $j>n$. A code of size M and minimum distance at least $2t+1$ has disjoint such balls, by the triangle inequality, and therefore

$$M\sum_{j=0}^{t}\binom{n}{j}\le 2^n.$$

Equality defines a perfect t-error-correcting code.

The bound constrains code size. Implementing the correction guarantee still requires an appropriate decoder.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.7 Linear codes and their matrices

A binary linear code is a subspace $C \leq \mathbb{F}_2^n$, with parameters $[n, k, d]$ and $|C| = 2^k$. A rank-k, $k \times n$ generator matrix encodes a row message u as $c = uG$.

A rank-$n - k$, $(n - k) \times n$ parity-check matrix satisfies

$$C = \{x : xH^T = 0\}, \qquad GH^T = 0.$$

Rank ensures that containment of the generator row space in this kernel is equality by dimension.

Row operations on G preserve the code but can change the message-to-codeword correspondence. Systematic form $[I_k \mid A]$ preserves message bits in the first k positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.7.1 Why orthogonality also needs rank

For $G = [I_k \mid A]$, take $H = [A^T \mid I_{n-k}]$ over $\mathbb{F}_2$. Then

$$GH^T = [I_k \mid A] \begin{bmatrix} A \\ I_{n-k} \end{bmatrix} = A + A = 0.$$

The identity blocks give ranks k and $n - k$. Orthogonality first proves $\operatorname{rowspan} G \subseteq \ker H$. Rank-nullity makes both spaces have dimension k, proving equality.

Without rank, $GH^T = 0$ is insufficient. For example, $G = [1\ 0]$ and $H = [0\ 0]$ satisfy it, but the row space of G is $\{00, 10\}$, while the kernel of H contains all four words.

A general generator matrix can be put in systematic form by row operations and a coordinate permutation. The permutation relabels coordinates and preserves Hamming distance. It is a separate operation from row reduction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.7.2 Minimum distance equals minimum nonzero weight

For a linear code, differences remain codewords and Hamming distance is translation invariant:

$$d(c_1, c_2) = wt(c_1 - c_2).$$

Every nonzero difference is a nonzero codeword, and every nonzero codeword is the difference of itself and zero. Hence

$$d(C) = \min_{c \in C,\, c \neq 0} wt(c).$$

For $C = \{00000, 00011, 01101, 01110\}$, the first two nonzero generators sum to the third. Its nonzero weights are 2, 3, 3, so minimum distance is 2.

This reduction uses linear or suitable group-code structure. For a general nonlinear code, inspect distances between all pairs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.8 Syndromes classify cosets

Define the row syndrome $s(r) = rH^T$. Adding a codeword preserves it, since $cH^T = 0$. Conversely,

$$s(r_1) = s(r_2) \iff (r_1 - r_2)H^T = 0 \iff r_1 - r_2 \in C \iff r_1 + C = r_2 + C.$$

Thus syndromes and additive cosets correspond bijectively, with kernel C. Full row rank of H makes every $(n-k)$-bit syndrome attainable.

Changing the row basis of H preserves its kernel but changes syndrome coordinates. More precisely, if $H' = UH$ with U invertible, then $s' = r(H')^T = sU^T$. The zero-syndrome decision is unchanged, but the lookup table must be transformed too; its old bit interpretation cannot simply be reused.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.8.1 Standard arrays and nearest decoding

The first row of a standard array is C. Choose an unused minimum-weight word x as a leader and fill the next row with $x + C$; repeat until all cosets appear.

If received $r = c^* + x$ with a minimum-weight representative x of its coset, then for every $c \in C$, $r + c = x + (c^* + c)$ lies in the same coset. Minimality gives

$$d(c^*, r) = wt(x) \leq wt(r + c) = d(c, r).$$

The decoded $c^* = r - x$ is therefore a nearest codeword.

Equal-weight leaders can produce ties. Specify a deterministic choice; the standard array and resulting decoder need not be unique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.8.2 The $[3, 2]$ parity code

Take $C = \{000, 011, 101, 110\}$ and $H = [1\ 1\ 1]$. Its two cosets are

Coset	Syndrome
$\{000, 011, 101, 110\}$	0
$\{100, 111, 001, 010\}$	1

The nonzero coset has three minimum-weight leaders, 100, 010, 001. Every single-bit error has syndrome 1, so that syndrome cannot identify its position. Minimum distance 2 guarantees single-error detection, but not unique correction of every single error.

Possessing a coset table does not itself supply a single-error correction guarantee. The distances and leader ties determine the limit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.9 Hamming parity-check columns

Use every nonzero three-bit column once:

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}.$$

No zero column excludes weight-1 codewords. No repeated columns excludes weight-2 codewords. But columns 1, 2, 3 sum to $110 + 101 + 011 = 000$, so 1110000 is a weight-3 codeword. Thus $d = 3$. The last three columns form the identity, so rank is 3, dimension is 4, and the code has parameters $[7, 4, 3]$.

Permuting columns only relabels coordinates and preserves distances. Always retain the actual column order when locating an error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.9.1 Complete systematic encoding

A compatible generator is

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

For $u = 1010$, add generator rows 1 and 3 to obtain $c = 1010101$. Parity coordinates satisfy

$$c_5 = c_1 + c_2 + c_4, \quad c_6 = c_1 + c_3 + c_4, \quad c_7 = c_2 + c_3 + c_4.$$

Consequently $cH^T = 0$.

After correction, the first four bits recover the message. Syndrome decoding itself must use all seven positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.9.2 Decoding and perfectness

For $r = 1110101$, compute row syndrome $rH^T = (1, 0, 1)$. Its transpose matches column 2, indicating a single error in position 2. Flip that bit to obtain 1010101.

There are 16 codewords, each with a radius-1 ball of size $1 + 7 = 8$. Their disjoint balls contain $16 \cdot 8 = 128 = 2^7$ words, covering the whole space. The Hamming code is therefore perfect for one-error correction.

A multiple-error syndrome is a column sum and can equal a single column. Interpreting it as one error is justified only within the promised error radius.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.9.3 A bit-by-bit syndrome trace

The transmitted word $c = 1010101$ becomes $r = 1110101$ after position 2 flips. The three checks, all modulo 2, give

$$s_1 = r_1 + r_2 + r_4 + r_5 = 1 + 1 + 0 + 1 = 1,$$

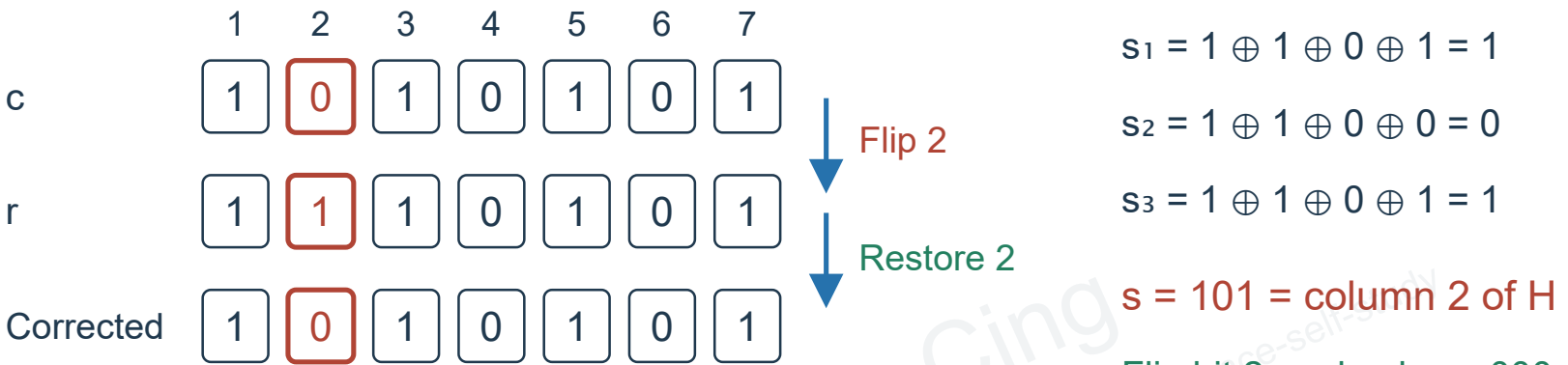
$$s_2 = r_1 + r_3 + r_4 + r_6 = 1 + 1 + 0 + 0 = 0, \qquad s_3 = r_2 + r_3 + r_4 + r_7 = 1 + 1 + 0 + 1 = 1.$$

The syndrome 101 equals column 2. Its numerical binary value 5 is not the error position, because this matrix's columns are not in numerical order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.9.3 A bit-by-bit syndrome trace (continued)

Position 2 flips: match the syndrome to a column, not its binary value



$s_1 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$

$s_2 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$

$s_3 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$

$s = 101$ = column 2 of H

Flip bit 2; recheck s = 000.

Top labels are bit positions. Red boxes mark position 2. All three sums are modulo 2.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.9.4 Erasures and the Singleton bound

An erasure has a known position. A bit flip does not. For erased positions E, move the known received coordinates to the right-hand side and solve

$$H_E c_E^T = b.$$

Unique recovery holds exactly when the unknown-coordinate columns are linearly independent. If $|E| \leq d - 1$, a column dependence would create a nonzero codeword of weight below d, a contradiction. Thus every set of at most $d - 1$ erasures can be recovered.

For a code of size M, alphabet size q, length n and finite minimum distance d, delete any $d - 1$ coordinates. Projection remains injective. Otherwise two distinct codewords could differ only in the deleted coordinates and have distance at most $d - 1$. Hence

$$M \leq q^{n-d+1}.$$

This is the Singleton bound. A linear $[n, k, d]$ code satisfies $k \leq n - d + 1$. Its proof differs from the sphere-packing argument.

16.10 Actions, orbits, and stabilizers

A finite group acts on a finite set X if $e \cdot x = x$ and $(gh) \cdot x = g \cdot (h \cdot x)$. Define

$$\operatorname{Orb}(x) = \{g \cdot x : g \in G\}, \quad \operatorname{Stab}(x) = \{g : g \cdot x = x\}.$$

The stabilizer is a subgroup: the identity fixes x, and if g, h fix it, so does gh^{-1}. The relation $y = g \cdot x$ is reflexive, symmetric and transitive by identity, inverses and composition, so orbits partition X. The map $g \operatorname{Stab}(x) \mapsto g \cdot x$ is onto the orbit and well defined; equal images mean $g_2^{-1} g_1$ fixes x, hence equal cosets. Thus

$$|G| = |\operatorname{Orb}(x)| \, |\operatorname{Stab}(x)|.$$

An orbit contains configurations; a stabilizer contains group elements. Their cardinalities enter different parts of the theorem.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.10.1 From positions to colorings

Suppose G permutes a position set Ω. For colorings $f : \Omega \to \{1, \dots, q\}$, define

$$(g \cdot f)(i) = f(g^{-1} i).$$

The color originally at i moves to gi. The inverse is needed because

$$((gh) \cdot f)(i) = f(h^{-1} g^{-1} i) = (g \cdot (h \cdot f))(i).$$

A coloring fixed by g must be constant on each cycle, so every cycle permits one independent color choice. The group moves positions and leaves color names fixed.

Burnside also applies to invariant subsets such as colorings with exactly three red positions. For proper colorings, adjacent vertices must differ, so fixed counts generally require adjacency constraints in addition to $q^{\#\text{cycles}}$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.10.2 Burnside by counting pairs

Let $F = \{(g, x) : g \cdot x = x\}$ and $X^g = \{x : g \cdot x = x\}$. Counting by g gives $|F| = \sum_g |X^g|$. Counting by x gives $|F| = \sum_x |\operatorname{Stab}(x)|$.

On one orbit O, orbit-stabilizer gives each stabilizer size $|G|/|O|$, so

$$\sum_{x \in O} |\operatorname{Stab}(x)| = |O| \frac{|G|}{|O|} = |G|.$$

Every orbit contributes $|G|$, yielding

$$\#(X/G) = \frac{1}{|G|} \sum_{g \in G} |X^g|.$$

The denominator is the group order, not the configuration count. Unequal orbit sizes are handled by their corresponding stabilizers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.10.3 Two-color square vertices

For four vertices with two named colors, there are 16 raw colorings. The square symmetry group D_4 has eight elements:

Motion	Number	Fixed colorings per motion
Identity	1	16
Quarter turns	2	2
Half turn	1	4
Opposite-vertex reflections	2	8
Opposite-edge reflections	2	4

Thus $(16 + 2 + 2 + 4 + 8 + 8 + 4 + 4)/8 = 6$ orbits. Rotations alone give $(16 + 2 + 2 + 4)/4 = 6$ as well.

The equality for these two groups is specific to this example. Adding reflections can merge orbits in other color sets.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.11 Cycle index

For a permutation g on n positions, let $j_k(g)$ count its length-k cycles. Define

$$Z_G(x_1, \dots, x_n) = \frac{1}{|G|} \sum_{g \in G} \prod_{k=1}^{n} x_k^{j_k(g)}.$$

For natural vertex actions,

$$Z_{C_3} = \frac{x_1^3 + 2x_3}{3}, \qquad Z_{C_4} = \frac{x_1^4 + x_2^2 + 2x_4}{4},$$

$$Z_{D_4} = \frac{x_1^4 + x_2^2 + 2x_4 + 2x_1^2 x_2 + 2x_2^2}{8}.$$

The cycle index describes an action before colors are chosen. Equal group orders do not imply equal cycle indices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.11.1 A fixed count and an orbit count

Let $c(g) = \sum_k j_k(g)$. A coloring fixed by one specified g is constant on each cycle, so

$$|X^g| = q^{c(g)} = \prod_k q^{j_k(g)}.$$

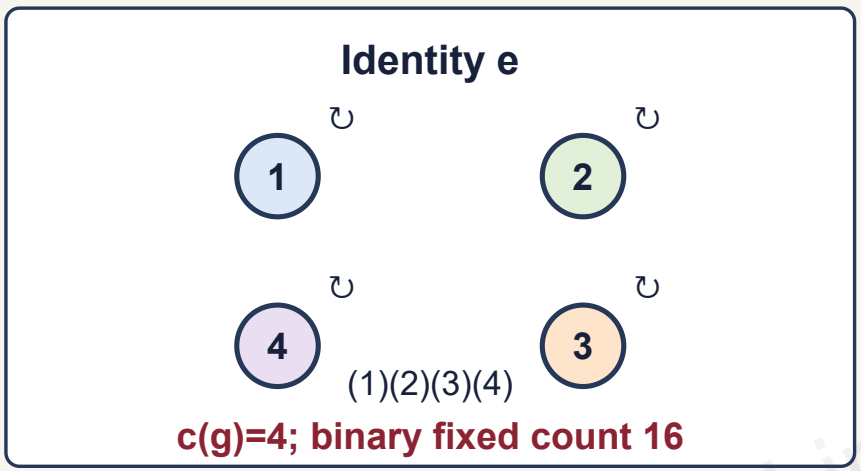
This is one element's fixed count. Only substituting into the already averaged cycle index gives the orbit count

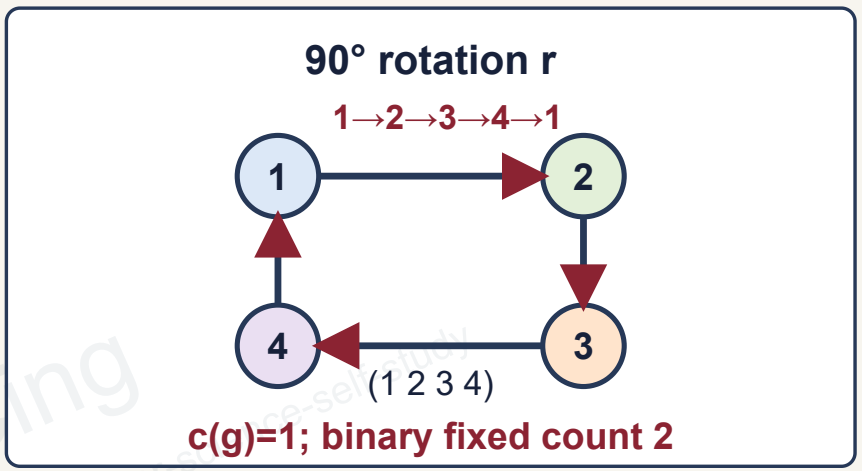
$$Z_G(q, \ldots, q) = \frac{1}{|G|} \sum_g q^{c(g)} = |X/G|.$$

The identity fixes 16 binary configurations, whereas a quarter turn fixes two.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.11.1 A fixed count and an orbit count (continued)

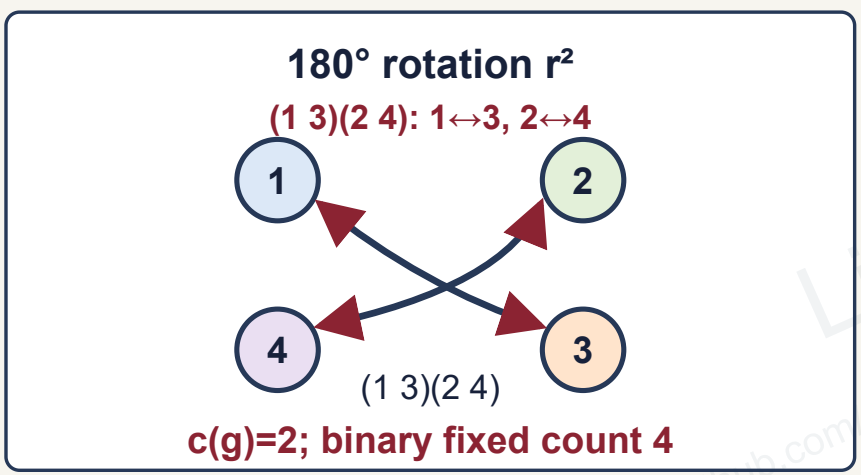


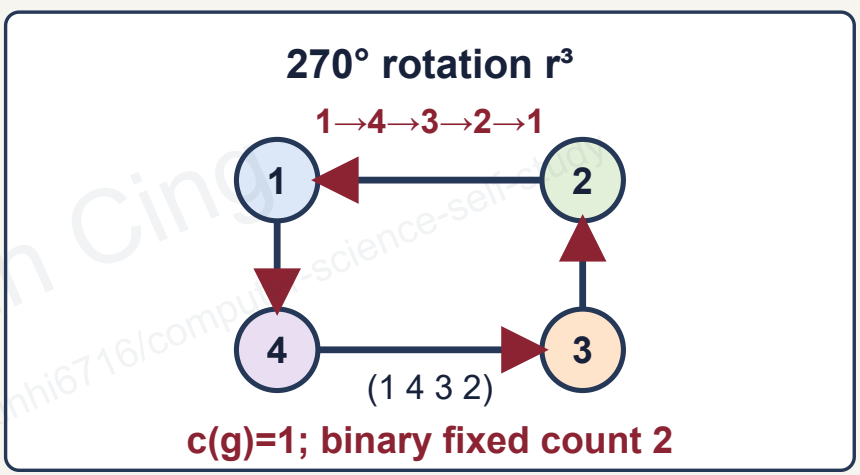


Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.11.2 Half and three-quarter turns

The half turn has two 2-cycles, shown by the double-headed arrows $1 \leftrightarrow 3$ and $2 \leftrightarrow 4$. The three-quarter turn has one 4-cycle. Their fixed binary counts are four and two. The diagram also adds all four counts before averaging.





4
rotations

16+2+4+2
= 24
24/4 = 6

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.11.3 The fixed-count table and group average

Rotation g	Cycles	$c(g)$	Fixed binary colorings $2^{c(g)}$
e	$(1)(2)(3)(4)$	4	16
r	(1234)	1	2
r^2	$(13)(24)$	2	4
r^3	(1432)	1	2

The group average is $(16+2+4+2)/4 = 6$. Representatives 0000, 0001, 0011, 0101, 0111, 1111 have orbit sizes 1, 4, 4, 2, 4, 1, summing to 16. The identity's fixed count 16 differs from the orbit count 6.

$$Z_{C_4}(q,q,q,q) = \frac{q^4+q^2+2q}{4}, \qquad Z_{D_4}(q,q,q,q) = \frac{q^4+3q^2+2q^3+2q}{8}.$$

For three colors, C_4 gives $(81+9+6)/4 = 24$ orbits and D_4 gives 21.

16.11.4 The separate three-position necklace

For binary colorings of three positions under C_3, the fixed counts are

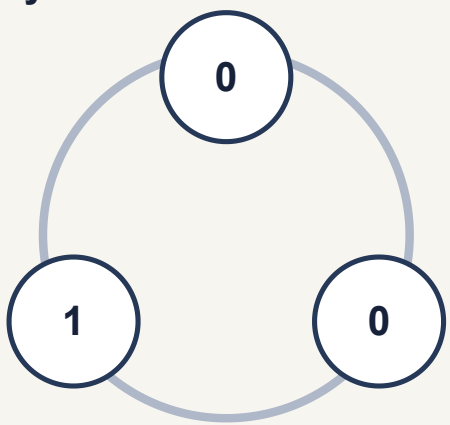
$$|X^e| = 2^3 = 8, \qquad |X^r| = |X^{r^2}| = 2,$$

so $|X/C_3| = (8 + 2 + 2)/3 = 4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.11.4 The separate three-position necklace (continued)

C3 binary necklace: fixed counts and orbits



Representatives: 000, 111, one 1, two 1s

(8 + 2 + 2) / 3 = 4

The identity fixes 8

Each nonidentity rotation fixes 2

The group average gives 4 orbits

Fixed counts and orbit counts differ

For the preceding binary square under C_4, the average is instead $(16 + 2 + 4 + 2)/4 = 6$. The configuration sets and groups differ. The shared fixed-count averaging method does not make their numerical counts or cycle decompositions interchangeable.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16.12 Polya substitution

Give colors formal weights $w_1, \ldots, w_m$. A constant color on a length-k cycle contributes w_i^k, so substitute

$$x_k \longmapsto \sum_{i=1}^{m} w_i^k.$$

The resulting pattern inventory counts inequivalent colorings by their color multiplicities. A monomial's coefficient is the number of orbits with those multiplicities. Setting every weight to 1 recovers the total orbit count.

Substituting the same unpowered sum for every x_k loses multiplicity information. The exponent records the number of positions occupied by that cycle's color.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.12.1 Proof of the weighted substitution

Give a coloring f weight $W(f) = \prod_{i \in \Omega} w_{f(i)}$. Since the group permutes positions, every coloring in an orbit O has the same monomial $W(O)$. Weighted double counting of fixed pairs gives

$$\frac{1}{|G|} \sum_{g} \sum_{f : g \cdot f = f} W(f) = \sum_{O} W(O).$$

Indeed, each of the $|O|$ colorings is fixed by $|G|/|O|$ elements, so averaging contributes its orbit monomial once.

For a fixed g, a length-k cycle in color i contributes w_i^k. Independent choices on different cycles give

$$\sum_{f : g \cdot f = f} W(f) = \prod_{k} \left(\sum_{i} w_i^k \right)^{j_k(g)}.$$

Substituting this expression into the average proves the cycle-index substitution, including the interpretation of every coefficient as an orbit count.

16.12.2 Three colors on a three-position necklace

For rotations C_3, colors r, w, b give

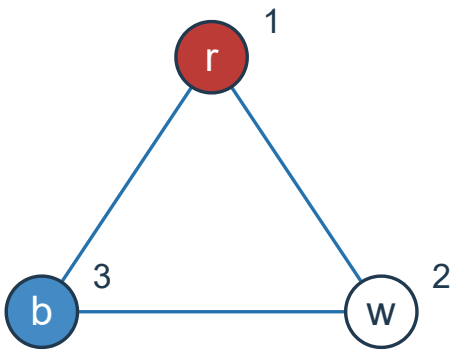
$$\frac{(r+w+b)^3 + 2(r^3+w^3+b^3)}{3} = r^3 + w^3 + b^3 + r^2w + r^2b + w^2r + w^2b + b^2r + b^2w + 2rwb.$$

The coefficient 2 on rwb counts two orientations of one red, one white, and one blue bead. Rotations preserve their cyclic order, so reversing it creates another orbit. Total coefficient sum is $3 + 6 + 2 = 11$, matching Burnside's $(27 + 3 + 3)/3$.

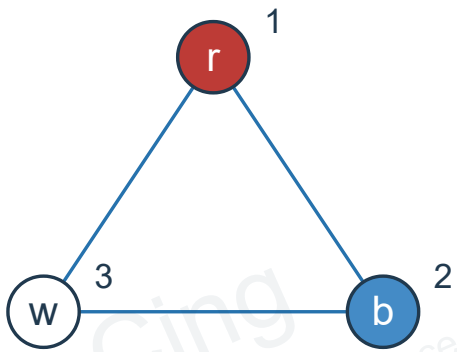
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.12.2 Three colors on a three-position necklace (continued)

One of each color: two rotation orbits; reflections merge them



Representative rwb



Representative rbw

Read colors around 1→2→3

Left: rwb, wbr, brw

Right: rbw, bwr, wrb

The two rotation lists are disjoint.

Reflection swaps positions 2 and 3 and exchanges the drawings. This motion is absent from C_3.

Allowing reflections changes the group and merges the two orientations. A necklace under rotations and a bracelet under rotations and reflections are different orbit problems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.12.3 Six positions with exactly three red beads

The rotational cycle index is

$$Z_{C_6} = \frac{x_1^6 + x_2^3 + 2x_3^2 + 2x_6}{6}.$$

Substitute $x_k = r^k + b^k$ and extract $r^3 b^3$. The four terms contribute $\binom{6}{3} = 20, 0, 2 \cdot 2 = 4, 0$, respectively. Thus there are $(20 + 4)/6 = 4$ rotational orbits. Dividing all $\binom{6}{3} = 20$ configurations by six fails because the alternating word $rbrbrb$ has only two distinct rotations.

Adding reflections to obtain D_6 adds three elements of type $1^2 2^2$ and three of type 2^3. For the former,

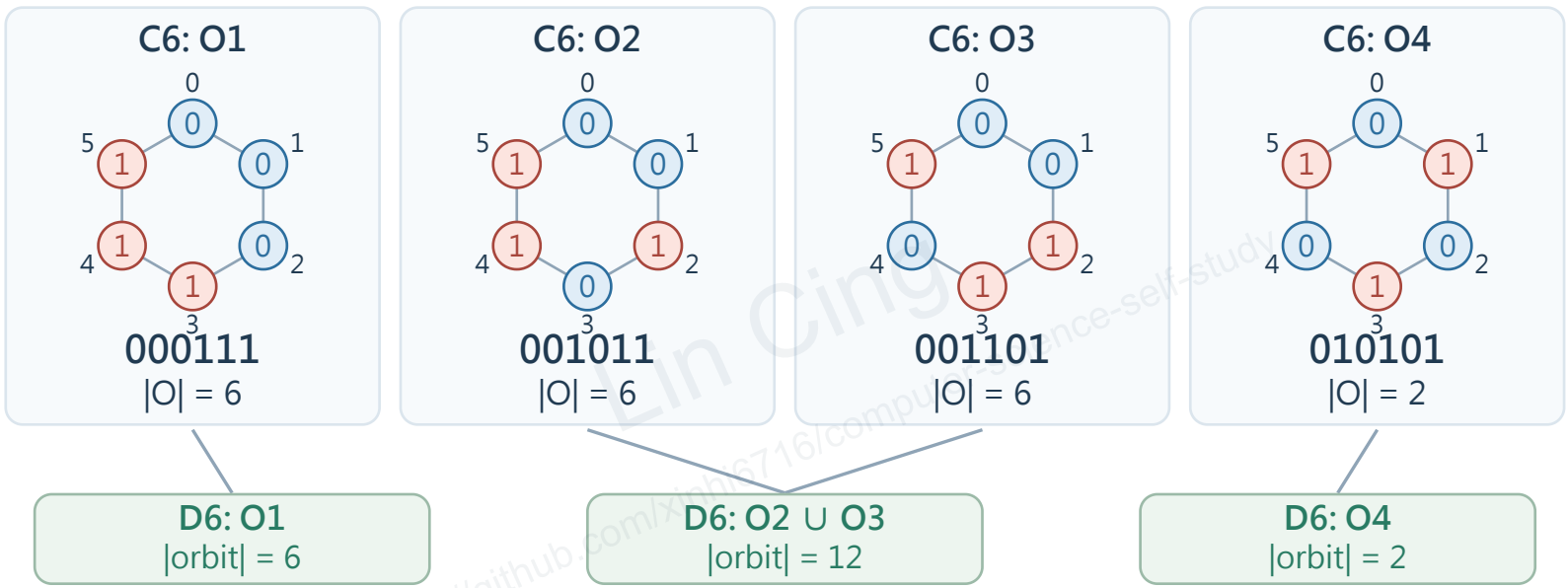
$$[r^3 b^3](r + b)^2 (r^2 + b^2)^2 = 2 \cdot 2 = 4.$$

The latter allow only even red multiplicities and contribute zero. The bracelet count is $(24 + 3 \cdot 4)/12 = 3$. Changing the permitted reflections can merge equivalence classes.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Four necklaces become three bracelets

Each card shows one representative, not every member of its orbit. Read the six bits clockwise from position 0; r sends position i to i+1 modulo 6.



0 = blue · 1 = red · positions 0–5 run clockwise

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Which two rotational classes merge?

For reflection $s: i \mapsto -i \pmod 6$, the word 001011 becomes 011010. A subsequent rotation r gives 001101. Thus reflections merge O2 and O3, which rotations alone cannot relate. O1 and O4 each remain a single class when reflections are added.

Group	Orbit sizes	Check against the 20 labeled configurations
C6	6, 6, 6, 2	6+6+6+2=20
D6	6, 12, 2	6+12+2=20

The alternating word 010101 is fixed by shifts 0,2,4. Its C6 stabilizer has size 3, so its orbit has size 6/3=2. This is why dividing all 20 configurations by 6 fails. Counting orbits uses the specified group action; adding reflections changes equivalence, not the underlying set of configurations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.13 Connections and checks

Quotient groups require normality for representative independence. RSA correctness spans the full message space through Fermat and CRT. Code distance and disjoint balls establish correction guarantees, while syndromes identify cosets. Burnside counts fixed pairs; cycle indices encode the action and Polya substitution records color weights.

For each exercise, state the group, code, or action before calculating. Full derivations and executable checks support the finite examples.

Keep hypotheses, counterexamples, and proofs with numerical answers. Memorizing orbit counts or decoding tables does not establish their applicability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study